
Operating Systems

Sheet 10 Solutions

Processor Scheduling

Name: Yehia Said Gewily

ID: 21010705

Date: May 2026

1 Conceptual Questions

1. Round-Robin Quantum Size

Argument for a small quantum:

A small quantum gives each process frequent turns on the CPU, resulting in better **response time** and a smoother interactive experience. Short processes finish quickly without waiting behind long ones. This is ideal for **interactive / time-sharing systems** with many short, bursty jobs.

Argument for a large quantum:

A large quantum reduces the number of context switches, decreasing **overhead**. More CPU time is spent doing useful work. This is ideal for **batch systems** with long, CPU-bound jobs where throughput matters more than response time.

Comparison:

- **Small quantum** → interactive systems, I/O-bound jobs, many users.
- **Large quantum** → batch systems, CPU-bound jobs, throughput-oriented.
- **Both reasonable:** General-purpose systems that handle a mix of interactive and batch work. The quantum should be large enough to amortize context-switch cost but small enough to keep response time acceptable.

2. Multilevel Feedback Queue — Favored Process Type

I/O-bound processes are generally favored.

Reason: I/O-bound processes use only a small fraction of their quantum before voluntarily blocking for I/O. Because they don't exhaust their quantum, they are **not demoted** to lower-priority queues. They remain in the higher-priority queues and get scheduled more often.

CPU-bound processes, in contrast, use their entire quantum, get demoted to lower-priority queues with longer quanta, and are scheduled less frequently.

3. Different Quantum Sizes at Different Levels

Advantage: It combines responsiveness with efficiency.

- **Higher-priority queues** have **small quanta**: this gives short/interactive processes fast turnaround and good response time.
- **Lower-priority queues** have **large quanta**: long CPU-bound processes that have been demoted can run for longer stretches, reducing context-switch overhead.

This way, the scheduler adapts: short jobs finish quickly at the top, while long jobs eventually get large uninterrupted CPU chunks at the bottom — balancing fairness, responsiveness, and throughput.

4. HRRN (Highest Response Ratio Next)

HRRN selects the process with the highest **response ratio**:

$$\text{Response Ratio} = \frac{W + S}{S}$$

where W = waiting time and S = expected service time.

a. How does it prevent starvation?

As a process waits longer, its W increases, which increases its response ratio. Even a very long process will eventually accumulate enough waiting time to have the highest ratio. Therefore, **no process can starve** — it will eventually be selected.

b. How does it decrease favoritism to short new processes?

Pure SJF always favors short processes regardless of how long other processes have waited. In HRRN, the waiting time W in the numerator acts as an “aging” mechanism. A long process that has waited a while can have a higher ratio than a short process that just arrived. This **balances** short-job preference with fairness to long-waiting jobs.

c. Two processes waiting about the same time — same priority?

No. If two processes have the same waiting time W , then:

$$R_1 = \frac{W + S_1}{S_1}, \quad R_2 = \frac{W + S_2}{S_2}$$

The process with the **shorter service time** will have a **higher** response ratio. For example, if $W = 10$, $S_1 = 2$, $S_2 = 10$: $R_1 = 6.0$ vs. $R_2 = 2.0$. The shorter process is still preferred, but by a diminishing margin as W grows.

5. Multilevel Queues — Scheduling Goals

a. Favor short processes

New processes enter the **highest-priority queue** with a small quantum. Short processes complete within that quantum and finish quickly without ever being demoted. Long processes exhaust their quantum and are pushed to lower-priority queues, effectively giving short processes priority.

b. Favor I/O-bound processes to improve I/O device utilization

I/O-bound processes voluntarily give up the CPU before their quantum expires (to do I/O), so they are **not demoted**. They stay in higher-priority queues and are scheduled frequently. This keeps I/O devices busy because the I/O-bound processes get to issue I/O requests without long waits.

c. Determine the nature of a process quickly

A new process starts at the top queue. If it uses its full quantum, it is likely CPU-bound and gets demoted. If it blocks early for I/O, it is likely I/O-bound and stays high. Within just **one or two quanta**, the scheduler has effectively classified the process and scheduled it accordingly.

2 Scheduling Computations

6. Scheduling a Given Workload

Workload:

Process	Burst Time	Priority	Arrival Time
P1	50 ms	4	0 ms
P2	20 ms	1	20 ms
P3	100 ms	3	40 ms
P4	40 ms	2	60 ms

a-i. Shortest Remaining Time (SRT — preemptive)

- $t = 0$: P1 arrives (rem=50). Run P1.
- $t = 20$: P2 arrives (rem=20). P1 rem=30. $20 < 30 \Rightarrow$ preempt P1, run P2.
- $t = 40$: P2 finishes. P3 arrives (rem=100). P1 rem=30. $30 < 100 \Rightarrow$ run P1.
- $t = 60$: P4 arrives (rem=40). P1 rem=10. $10 < 40 \Rightarrow$ continue P1.
- $t = 70$: P1 finishes. P4 rem=40, P3 rem=100. Run P4.
- $t = 110$: P4 finishes. Run P3.
- $t = 210$: P3 finishes.

Time	0–20	20–40	40–70	70–110	110–210
Process	P1	P2	P1	P4	P3

a-ii. Non-Preemptive Priority (smaller = higher)

Priorities: P2=1, P4=2, P3=3, P1=4.

- $t = 0$: Only P1 available. Run P1 (non-preemptive).
- $t = 50$: P1 finishes. Available: P2 (pri=1), P3 (pri=3). Run P2.
- $t = 70$: P2 finishes. Available: P3 (pri=3), P4 (pri=2). Run P4.
- $t = 110$: P4 finishes. Run P3.
- $t = 210$: P3 finishes.

Time	0–50	50–70	70–110	110–210
Process	P1	P2	P4	P3

a-iii. Round Robin (quantum = 30 ms)

- $t = 0$: P1 starts. $t = 20$: P2 arrives (queued). $t = 30$: P1 quantum expires (rem=20). Ready: [P2, P1].
- $t = 30$: Run P2. $t = 40$: P3 arrives (queued). $t = 50$: P2 finishes. Ready: [P1, P3].
- $t = 50$: Run P1 (rem=20). $t = 60$: P4 arrives (queued). $t = 70$: P1 finishes. Ready: [P3, P4].
- $t = 70$: Run P3 (rem=100). $t = 100$: quantum expires (rem=70). Ready: [P4, P3].

- $t = 100$: Run P4 (rem=40). $t = 130$: quantum expires (rem=10). Ready: [P3, P4].
- $t = 130$: Run P3 (rem=70). $t = 160$: quantum expires (rem=40). Ready: [P4, P3].
- $t = 160$: Run P4 (rem=10). $t = 170$: P4 finishes. Ready: [P3].
- $t = 170$: Run P3 (rem=40). $t = 200$: quantum expires (rem=10). Ready: [P3].
- $t = 200$: Run P3 (rem=10). $t = 210$: P3 finishes.

Time	0–30	30–50	50–70	70–100	100–130	130–160	160–170	170–200	200–210
Proc	P1	P2	P1	P3	P4	P3	P4	P3	P3

b. Average Waiting Times

Waiting time = Completion time – Arrival time – Burst time.

Policy	P1	P2	P3	P4		Av
SRT	$70 - 0 - 50 = 20$	$40 - 20 - 20 = 0$	$210 - 40 - 100 = 70$	$110 - 60 - 40 = 10$		$\frac{100}{4}$
Priority	$50 - 0 - 50 = 0$	$70 - 20 - 20 = 30$	$210 - 40 - 100 = 70$	$110 - 60 - 40 = 10$		$\frac{110}{4}$
RR(30)	$70 - 0 - 50 = 20$	$50 - 20 - 20 = 10$	$210 - 40 - 100 = 70$	$170 - 60 - 40 = 70$		$\frac{170}{4}$

7. Optimal and Pessimal Workloads

a. FIFO (First In, First Out)

- **Optimal:** When all jobs arrive at the same time and are already sorted **shortest first** (equivalent to SJF). Also optimal when there is only one job or all jobs have equal length.
- **Pessimal:** When all jobs arrive at the same time and are sorted **longest first** — short jobs must wait behind long ones, maximizing average response time.

b. Round Robin

- **Optimal:** When all jobs have exactly **equal length** (no wasted context switches; equivalent to any other policy). Also approaches optimal for workloads where all jobs are very short (finish within one quantum).
- **Pessimal:** When there are many jobs of varying lengths and the quantum is very small relative to job sizes, causing excessive context-switch overhead without improving ordering.

c. Shortest Time to Completion First (STCF, preemptive)

- **Optimal:** This policy is **provably optimal** for minimizing average response time for **all workloads** (when service times are known). It always produces the minimum average response time.
- **Pessimal:** There is **no workload** under which STCF is pessimal — it is never the worst policy.

d. Multilevel Feedback

- **Optimal:** When the workload is a mix of short and long jobs with unknown service times. MLFQ approximates STCF without needing advance knowledge of burst lengths.
- **Pessimal:** When all jobs are CPU-bound and have the same length — the overhead of multiple queue levels and demotions adds unnecessary context switches with no scheduling benefit.

8. Scheduling with Context-Switch Overhead

Workload:

Job	Arrival	Service
1	0	4
2	1	8
3	3	2
4	10	6
5	12	5

Context-switch overhead = 1 time unit. Total service

= 25.

a. FCFS

Time	0–4	4–5	5–13	13–14	14–16	16–17	17–23	23–24	24–29
	J1	CS	J2	CS	J3	CS	J4	CS	J5

Job	Completion	TAT	NTAT (TAT/S)	Wait
1	4	4	1.00	0
2	13	12	1.50	4
3	16	13	6.50	11
4	23	13	2.17	7
5	29	17	3.40	12
Average TAT		11.8		

Processor efficiency = $25/29 = 86.2\%$.

b. SPN (Shortest Process Next — non-preemptive)

Time	0–4	4–5	5–7	7–8	8–16	16–17	17–22	22–23	23–29
	J1	CS	J3	CS	J2	CS	J5	CS	J4

At $t = 4$: J2(8) and J3(2) available $\rightarrow$ pick J3. At $t = 7$: only J2. At $t = 16$: J4(6) and J5(5) $\rightarrow$ pick J5.

Job	Completion	TAT	NTAT	Wait
1	4	4	1.00	0
2	16	15	1.875	7
3	7	4	2.00	2
4	29	19	3.167	13
5	22	10	2.00	5
Average TAT		10.4		

Processor efficiency = $25/29 = 86.2\%$.

c. SRTN (Shortest Remaining Time Next — preemptive)

- $t = 0$: J1 (rem=4). $t = 1$: J2 arrives (rem=8), J1 rem=3. No preempt.
- $t = 3$: J3 arrives (rem=2), J1 rem=1. No preempt. $t = 4$: J1 done. CS.
- $t = 5$: J3(2) vs J2(8). Run J3. $t = 7$: J3 done. CS.
- $t = 8$: Run J2. $t = 10$: J4(6), J2 rem=6 (tie, continue). $t = 12$: J5(5), J2 rem=4. Continue.
- $t = 16$: J2 done. CS. $t = 17$: J5(5) vs J4(6). Run J5.
- $t = 22$: J5 done. CS. $t = 23$: Run J4. $t = 29$: J4 done.

No preemptions occur — same schedule as SPN for this workload.

Job	Completion	TAT	NTAT	Wait
1	4	4	1.00	0
2	16	15	1.875	7
3	7	4	2.00	2
4	29	19	3.167	13
5	22	10	2.00	5
Average TAT		10.4		

Processor efficiency = $25/29 = 86.2\%$.

d. Round Robin, quantum = 3

Trace (CS = context switch of 1 unit):

- $t = 0$: J1 starts (rem=4).

- $t = 1$: J2 arrives $\rightarrow$ ready queue.
- $t = 3$: J1 quantum expires (rem=1). J3 arrives. CS. Ready: [J2, J3, J1].
- $t = 4$: J2 runs. $t = 7$: quantum expires (rem=5). CS. Ready: [J3, J1, J2].
- $t = 8$: J3 runs. $t = 10$: J3 finishes (service=2). J4 arrives. CS. Ready: [J1, J2, J4].
- $t = 11$: J1 runs (rem=1). $t = 12$: J1 finishes. J5 arrives. CS. Ready: [J2, J4, J5].
- $t = 13$: J2 runs (rem=5). $t = 16$: quantum expires (rem=2). CS. Ready: [J4, J5, J2].
- $t = 17$: J4 runs (rem=6). $t = 20$: quantum expires (rem=3). CS. Ready: [J5, J2, J4].
- $t = 21$: J5 runs (rem=5). $t = 24$: quantum expires (rem=2). CS. Ready: [J2, J4, J5].
- $t = 25$: J2 runs (rem=2). $t = 27$: J2 finishes. CS. Ready: [J4, J5].
- $t = 28$: J4 runs (rem=3). $t = 31$: J4 finishes. CS. Ready: [J5].
- $t = 32$: J5 runs (rem=2). $t = 34$: J5 finishes.

Job	Completion	TAT	NTAT	Wait
1	12	12	3.00	8
2	27	26	3.25	18
3	10	7	3.50	5
4	31	21	3.50	15
5	34	22	4.40	17
Average TAT		17.6		

Context switches = 9. Processor efficiency = $25/34 = 73.5\%$.

e. Multilevel Feedback Queue (quantum = 2^i at level i)

Level 1: $q = 2$, Level 2: $q = 4$, Level 3: $q = 8$, ...

Processes start at Level 1. After using a full quantum, they are demoted. Higher-level queues preempt lower ones.

- $t = 0$: J1 $\rightarrow$ Q1 ($q = 2$). Run J1. $t = 1$: J2 arrives $\rightarrow$ Q1.
- $t = 2$: J1 quantum done (rem=2), J1 $\rightarrow$ Q2. CS. $t = 3$: Run J2 from Q1. J3 arrives $\rightarrow$ Q1.
- $t = 5$: J2 quantum done (rem=6), J2 $\rightarrow$ Q2. CS. $t = 6$: Run J3 from Q1 ($q = 2$).
- $t = 8$: J3 finishes (service=2). CS.

- $t = 9$: Q1 empty. Q2: [J1(rem=2), J2(rem=6)]. Run J1 ($q = 4$).
- $t = 10$: J4 arrives $\rightarrow$ Q1. Preempt J1 (ran 1, rem=1, stays in Q2). CS.
- $t = 11$: Run J4 from Q1. $t = 12$: J5 arrives $\rightarrow$ Q1.
- $t = 13$: J4 quantum done (rem=4), J4 $\rightarrow$ Q2. CS. $t = 14$: Run J5 from Q1.
- $t = 16$: J5 quantum done (rem=3), J5 $\rightarrow$ Q2. CS.
- $t = 17$: Q1 empty. Q2: [J1(rem=1), J2(rem=6), J4(rem=4), J5(rem=3)]. Run J1 ($q = 4$).
- $t = 18$: J1 finishes (rem=1). CS.
- $t = 19$: Run J2 from Q2 ($q = 4$). $t = 23$: J2 quantum done (rem=2), J2 $\rightarrow$ Q3. CS.
- $t = 24$: Run J4 from Q2 ($q = 4$). $t = 28$: J4 finishes. CS.
- $t = 29$: Run J5 from Q2 ($q = 4$). $t = 32$: J5 finishes (rem=3). CS.
- $t = 33$: Q2 empty. Q3: [J2(rem=2)]. Run J2 ($q = 8$). $t = 35$: J2 finishes.

Job	Completion	TAT	NTAT	Wait
1	18	18	4.50	14
2	35	34	4.25	26
3	8	5	2.50	3
4	28	18	3.00	12
5	32	20	4.00	15
Average TAT		19.0		

Context switches = 10. Processor efficiency = $25/35 = 71.4\%$.

9. Five Batch Jobs

Given: All arrive at time 0. No process-switching overhead.

Job	Running Time	Priority (lower = higher)
A	15	6
B	9	3
C	3	7
D	6	9
E	12	4

Total = 45 min.

a. Round Robin ($q = 1$ minute)

All 5 processes rotate. Each gets 1 minute per round.

- C (burst=3) finishes at minute $3 \times 5 - 2 = 13$.
- D (burst=6) finishes at $5 \times 3 + 4 \times 3 - 1 = 26$.
- B (burst=9) finishes at $5 \times 3 + 4 \times 3 + 3 \times 3 - 1 = 35$.
- E (burst=12) finishes at $5 \times 3 + 4 \times 3 + 3 \times 3 + 2 \times 3 = 42$.
- A (burst=15) finishes at 45.

Job	Turnaround
A	45
B	35
C	13
D	26
E	42
Average	$(45 + 35 + 13 + 26 + 42)/5 = \mathbf{32.2}$ min

b. Priority Scheduling (non-preemptive, one at a time)

Order by priority: B(3), E(4), A(6), C(7), D(9).

Job	Start	End	Turnaround
B	0	9	9
E	9	21	21
A	21	36	36
C	36	39	39
D	39	45	45
Average			30.0 min

c. FCFS (order: A, B, C, D, E)

Job	Start	End	Turnaround
A	0	15	15
B	15	24	24
C	24	27	27
D	27	33	33
E	33	45	45
Average			28.8 min

d. Shortest Job First

Order: C(3), D(6), B(9), E(12), A(15).

Job	Start	End	Turnaround
C	0	3	3
D	3	9	9
B	9	18	18
E	18	30	30
A	30	45	45
Average			21.0 min

SJF gives the best average turnaround, as expected.

10. Three Processes with CPU and I/O Bursts

Workload:

Proc	CPU1	IO1	CPU2	IO2	CPU3	Arrival
P1	2	4	2	2	2	0
P2	2	2	3	3	1	1
P3	1	2	1	1	1	1

Rule: If I/O completion and CPU timeout coincide, treat I/O completion first.

a. FCFS

Detailed trace:

- $t = 0-2$: P1 runs CPU1. At $t = 1$: P2, P3 arrive. $t = 2$: P1 $\rightarrow$ IO1 (done at $t = 6$).
- $t = 2-4$: P2 runs CPU1. $t = 4$: P2 $\rightarrow$ IO1 (done at $t = 6$).
- $t = 4-5$: P3 runs CPU1. $t = 5$: P3 $\rightarrow$ IO1 (done at $t = 7$). CPU idle $t = 5-6$.
- $t = 6$: P1 and P2 IO complete. Ready: [P1, P2]. Run P1.
- $t = 6-8$: P1 runs CPU2. $t = 7$: P3 IO done, enters ready. $t = 8$: P1 $\rightarrow$ IO2 (done at $t = 10$).
- $t = 8-11$: P2 runs CPU2 (burst=3). $t = 10$: P1 IO done, enters ready. $t = 11$: P2 $\rightarrow$ IO2 (done at $t = 14$).
- $t = 11-12$: P3 runs CPU2 (burst=1). $t = 12$: P3 $\rightarrow$ IO2 (done at $t = 13$).
- $t = 12-14$: P1 runs CPU3 (burst=2). $t = 13$: P3 IO done. $t = 14$: P1 **DONE**.
- $t = 14$: P2 IO done. Ready: [P3, P2]. Run P3.
- $t = 14-15$: P3 runs CPU3 (burst=1). P3 **DONE**.
- $t = 15-16$: P2 runs CPU3 (burst=1). P2 **DONE**.

Gantt chart:

Time	0-2	2-4	4-5	5-6	6-8	8-11	11-12	12-14	14-15	15-16
CPU	P1	P2	P3	idle	P1	P2	P3	P1	P3	P2

Proc	Waiting	TAT	
P1	$14 - 0 - 6 - 6 = 2$	$14 - 0 = 14$	
P2	$16 - 1 - 6 - 5 = 4$	$16 - 1 = 15$	
P3	$15 - 1 - 3 - 3 = 8$	$15 - 1 = 14$	
Mean	4.67	14.33	

b. Round Robin ($q = 1$)

Detailed trace:

- $t = 0-1$: P1 runs (CPU1 rem=1). $t = 1$: P2, P3 arrive. Ready: [P2, P3, P1].
- $t = 1-2$: P2 runs (CPU1 rem=1). Ready: [P3, P1, P2].
- $t = 2-3$: P3 runs (CPU1 done, burst=1). P3 $\rightarrow$ IO1 (done $t = 5$). Ready: [P1, P2].
- $t = 3-4$: P1 runs (CPU1 done). P1 $\rightarrow$ IO1 (done $t = 8$). Ready: [P2].
- $t = 4-5$: P2 runs (CPU1 done). P2 $\rightarrow$ IO1 (done $t = 7$). $t = 5$: P3 IO done.
- $t = 5-6$: P3 runs (CPU2 done, burst=1). P3 $\rightarrow$ IO2 (done $t = 7$). CPU idle.
- $t = 6-7$: idle.
- $t = 7$: P2 IO1 done, P3 IO2 done. Ready: [P2, P3]. Run P2.
- $t = 7-8$: P2 runs (CPU2 rem=2). $t = 8$: P1 IO1 done $\rightarrow$ Ready: [P3, P1, P2].
- $t = 8-9$: P3 runs (CPU3 done, burst=1). P3 **DONE**. Ready: [P1, P2].
- $t = 9-10$: P1 runs (CPU2 rem=1). Ready: [P2, P1].
- $t = 10-11$: P2 runs (CPU2 rem=1). Ready: [P1, P2].
- $t = 11-12$: P1 runs (CPU2 done). P1 $\rightarrow$ IO2 (done $t = 14$). Ready: [P2].
- $t = 12-13$: P2 runs (CPU2 done). P2 $\rightarrow$ IO2 (done $t = 16$). CPU idle.
- $t = 13-14$: idle.
- $t = 14$: P1 IO2 done. Run P1.
- $t = 14-15$: P1 runs (CPU3 rem=1). Ready: [P1].
- $t = 15-16$: P1 runs (CPU3 done). P1 **DONE**. $t = 16$: P2 IO2 done.
- $t = 16-17$: P2 runs (CPU3 done, burst=1). P2 **DONE**.

Gantt chart:

t	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
CPU	P1	P2	P3	P1	P2	P3	–	P2	P3	P1	P2	P1	P2	–	P1	P1	P2

Proc	Completion	Waiting	TAT
P1	16	$16 - 0 - 6 - 6 = 4$	16
P2	17	$17 - 1 - 6 - 5 = 5$	16
P3	9	$9 - 1 - 3 - 3 = 2$	8
Mean		3.67	13.33

c. Round Robin ($q = 2$)

Detailed trace:

- $t = 0-2$: P1 runs CPU1 (burst=2, done). P1 $\rightarrow$ IO1 (done $t = 6$). At $t = 1$: P2,P3 arrive.
- $t = 2-4$: P2 runs CPU1 (burst=2, done). P2 $\rightarrow$ IO1 (done $t = 6$). Ready: [P3].
- $t = 4-5$: P3 runs CPU1 (burst=1, done). P3 $\rightarrow$ IO1 (done $t = 7$). Idle $t = 5-6$.
- $t = 6$: P1 and P2 IO done. Ready: [P1, P2]. Run P1.
- $t = 6-8$: P1 runs CPU2 (burst=2, done). P1 $\rightarrow$ IO2 (done $t = 10$). $t = 7$: P3 IO done.
- $t = 8-10$: P2 runs CPU2 (burst=3, quantum=2, rem=1). $t = 10$: quantum expires. P1 IO done.
- IO completion first: P1 enters ready. Ready: [P3, P1, P2]. Run P3.
- $t = 10-11$: P3 runs CPU2 (burst=1, done). P3 $\rightarrow$ IO2 (done $t = 12$). Ready: [P1, P2]. Run P1.
- $t = 11-13$: P1 runs CPU3 (burst=2, done). P1 **DONE**. $t = 12$: P3 IO done.
- $t = 13-14$: P2 runs CPU2 (rem=1, done). P2 $\rightarrow$ IO2 (done $t = 17$). Ready: [P3]. Run P3.
- $t = 14-15$: P3 runs CPU3 (burst=1, done). P3 **DONE**. Idle $t = 15-17$.
- $t = 17$: P2 IO done. Run P2.
- $t = 17-18$: P2 runs CPU3 (burst=1, done). P2 **DONE**.

Gantt chart:

Time	0-2	2-4	4-5	5-6	6-8	8-10	10-11	11-13	13-14
CPU	P1	P2	P3	idle	P1	P2	P3	P1	P2

14-15	15-17	17-18
P3	idle	P2

Proc	Completion	Waiting	TAT
P1	13	$13 - 0 - 6 - 6 = 1$	13
P2	18	$18 - 1 - 6 - 5 = 6$	17
P3	15	$15 - 1 - 3 - 3 = 8$	14
Mean		5.0	14.67

d. Summary

Algorithm	Avg. Waiting	Avg. TAT
FCFS	4.67	14.33
RR($q=1$)	3.67	13.33
RR($q=2$)	5.00	14.67

e. Part (c) — Priority with RR

P2 has highest priority (higher than P1 and P3).

Why not FCFS? FCFS runs processes in arrival order regardless of priority; priority has no effect, so it does not make sense to consider it.

With priority-based RR, whenever P2 is in the ready queue it is selected first. This primarily affects tie-breaking: P2 gets the CPU before P1 or P3 when they are all ready simultaneously.

For RR $q = 1$: The main difference from the non-priority trace is that at $t = 7$ (when P2 and P3 are both ready), P2 runs first instead of following strict FIFO order, and similarly at other tie points. The overall completion times shift slightly in P2's favor.

For RR $q = 3$: With a larger quantum, P2 gets longer uninterrupted runs when selected, which can significantly reduce P2's waiting time. However, P1 and P3 may wait longer as a result.